Introduction to Software Engineering

Project Proposal & Plan

The student team is required to complete the **Project Proposal & Plan** documentation for the assigned course project, following the attached template.



Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

Objectives **1**

1 Member Contribution Assessment **2**

2 Vision **3**

2.1 Preliminary Problem Statement 3

Operating Environment 4

Design and Implementation Constraints 5

Expected Outcome and Preliminary Scope 6

2.2 Product Position Statement 6

2.3 Stakeholders and Users 7

2.4 Scope and Key Features 8

3 Proposed Solution **10**

3.1 Preliminary Architecture 10

3.2 Technology Stack 11

3.3 Hardware and Operating Environment 13

1. Development Environment 13

2. Testing Environment 13

3. Production Environment 14

4 Project Plan **14**

4.1 Process and Phase Gates..... 14

4.2 Work Breakdown Structure (WBS)..... 15

1.0 Requirements and Planning Phase..... 15

2.0 System Architecture and Design Phase..... 16

3.0 Implementation Phase..... 16

4.0 Testing and Deployment Phase..... 17

4.3 Estimation..... 17

Three-Point Estimation:..... 17

4.4 Schedule..... 20

4.5 Roles and Responsibilities..... 20

4.6 Human Resources and Cost Estimate..... 21

5 Risk Management..... 23

6 Tools & Collaboration Setup..... 25

7 AI Usage Log..... 27

Project Proposal & Plan

Objectives

This document focus on the following topics:

✔️ Completing the Project Proposal & Plan document with the following sections:

- Vision
- Proposed Solution
- Project Plan
- Risk Management
- Tools & Collaboration Setup

✔️ Including an AI usage log for this phase

✔️ Understanding the Project Proposal & Plan document.

1 Member Contribution Assessment

[List every member. Contribution (%) must add up to 100%. Evidence: links to tasks on the board, commits or pull requests.]

ID	Name	Role(s)	Contribution (%)	Evidence
24127352	Nguyễn Lương Hoàng Duy	Team lead	25	Task board
24127343	Trần Phúc Đạt	Designer	25	Task board
24127346	Niê Ngọc Lâm	Developer	25	Taskboard
24127521	Nguyễn Việt Quang	Tester	25	Task board

2 Vision

2.1 Preliminary Problem Statement

[Provide an approximately 1–2 page business description of the software problem, including the operating environment (e.g., web browser supporting HTML5, cloud server) and design & implementation constraints (e.g., programming language, database, documentation standards). State the problem at a preliminary level; the detailed problem statement is presented in PA2.]

✔️ *Examples: https://hackmd.io/@nndkhoa9/se-onlineauction-spec*

✔️ *https://hackmd.io/@nndkhoa9/se-healthmonitor-spec*

Business Context and Problem Description

In modern academic and professional environments, individuals often need to manage multiple responsibilities within a single day. Students must balance lectures, assignments, examinations, group projects, and personal activities, while office workers and freelancers need to organize daily tasks and maintain productivity. Without an effective system for organizing these responsibilities, users may struggle to determine what needs to be done first, keep track of unfinished work, and evaluate their daily progress.

A common approach is to rely on mental notes, handwritten lists, or general-purpose productivity applications. However, mental notes can easily be forgotten, handwritten lists may be difficult to update, and comprehensive productivity platforms can introduce unnecessary complexity for users who only need a simple daily planning tool. Some users may also prefer to keep their task information on their own computers instead of depending on an online account or cloud-based service.

These limitations create a need for a lightweight, easy-to-use desktop application that allows users to organize daily tasks, identify priorities, and monitor their progress in one place. The proposed product, **WhatYaDoing?**, aims to address this need by providing essential task management and daily productivity features through a simple interface.

Intended Users and Their Needs

The primary target users are students who need to manage assignments, study schedules, and personal responsibilities. They require a straightforward way to record tasks, distinguish important activities from less urgent ones, and track how much work they have completed during the day.

The application may also benefit office workers and freelancers who need to structure their daily workload without using a complex project management platform. These users may value quick task entry, clear priority indicators, and an overview of their completed and unfinished activities.

Across these user groups, the main problem is not simply creating a task list, but maintaining a clear and accessible overview of daily responsibilities. WhatYaDoing? therefore focuses on reducing the effort required to organize tasks and making daily progress easier to understand.

Proposed Software Solution

WhatYaDoing? is an application designed to help users plan their days, prioritize tasks, and track daily progress. The application will allow users to create, edit, delete, and complete tasks. Users will also be able to assign priority levels to tasks, represented by visual color indicators, and filter their task lists by completion status or priority.

To help users evaluate their progress, the application will provide a visual indicator of daily task completion. Task information and application settings will be stored locally on the user's computer, allowing the application to operate without requiring a continuous Internet connection.

Additional features, such as a task timer, a daily summary, and keyword-based task search, may be considered if development time permits. These features are secondary to the core task management functionality.

Operating Environment

The proposed system is designed as a cross-platform application targeting desktop and mobile devices, with Windows and Android as the initial deployment platforms. React Native will be used to develop the user interface, while NativeWind will support consistent and responsive styling across supported screen sizes.

The back-end logic will be implemented using JavaScript and TypeScript with Express, providing the application's server-side functionality and API endpoints. Supabase will serve as the primary online data storage and synchronization platform, allowing users to access their task data across supported devices. Local storage may be used to support essential task management functionality when the application is offline.

GitHub Actions will be used to automate the build and packaging process for supported platforms, while Jest will support automated testing of core application functionality. The specific desktop integration and packaging approach will be finalized during the design and implementation stages.

Design and Implementation Constraints

The project will be developed by a team of four students within a limited academic schedule, with approximately three weeks allocated for implementing the core features. Therefore, the team must prioritize essential functionality and avoid unnecessary architectural complexity.

The implementation should remain consistent with the technology stack defined in Section 3.2, including React Native and NativeWind for the front end, JavaScript and TypeScript with Express for the back end, and Supabase for online data storage and synchronization. The team must also consider the additional effort required to support both desktop and mobile platforms within the available development time.

Because the application is expected to support offline access, the team must determine how local data will be stored and synchronized with Supabase when connectivity is restored. Appropriate handling of network failures, synchronization conflicts, and invalid or missing data will be necessary to reduce the risk of data loss.

The initial version is intended primarily for individual task management. Advanced collaborative features, complex permission systems, and integrations with external calendar services are outside the current scope unless time and resources permit.

The team must also follow the course's software engineering practices, including maintaining project documentation, reviewing code changes, coordinating through the team's communication channels, writing appropriate tests, and documenting significant AI assistance in accordance with the course policy.

Expected Outcome and Preliminary Scope

The expected outcome is a functional cross-platform task management application that enables users to manage daily tasks, organize them by priority, filter task lists, and monitor their completion progress.

The initial development scope prioritizes task creation, editing, and deletion; task prioritization and filtering; local and online data storage; synchronization through Supabase; and daily progress visualization. The application should provide essential task management functionality on supported platforms, with offline access to locally available data where implemented.

The main risks include underestimating cross-platform development complexity, encountering integration issues between the front end and back end, handling offline data synchronization, and dealing with limited team member availability. These risks will be mitigated by prioritizing must-have features, dividing responsibilities among team members, conducting regular progress reviews, testing core functionality incrementally, and postponing optional features when necessary.

2.2 Product Position Statement

Element	Content
For	Students, office workers and those who want to organize their daily activities.
Who	struggle with lack of daily structure, task prioritization, and cognitive overload when managing daily tasks
The (product name)	WhatYaDoing
That	Help users plan their days, prioritize tasks with color tasks, and

	track daily progress offline.
Unlike	online cloud-based task platforms like Microsoft To Do or complex tools like Todoist.
Our product	provides a simple, offline-first, local JSON data storage application that ensures privacy, low learning curve, and fast task tracking

2.3 Stakeholders and Users

#	Stakeholder / user	Description	Main interests
1	Course Instructor & TA	course instructors and teaching assistants evaluating the project	Ensuring software quality, proper implementation of Must-have features, project feasibility, and adherence to requirements within 3 weeks
2	Project Team Members (The 4 idiots)	Developers, designers, tester, and team lead building the application	Completing the project on time, meeting work standards, fair contribution, achieving target grade (7-8), and maintaining code quality
3	Students (End User)	College or school students managing academic deadlines, coursework, and daily personal	Free, fast, local task tracking, easy assignment management, and visual

		chores	progress tracking to balance academic workload
4	Office Workers & Freelancers (End User)	Professionals needing a clean, offline tool for immediate daily task dumping and priority management	High privacy (local storage), zero account setup, intuitive interface, priority color-coding, and offline availability

2.4 Scope and Key Features

[List the main features with a MoSCoW priority. Only Must-have features are required to be fully implemented in PA5. State clearly what is out of scope.]

ID	Feature	Description	Priority (M/S/C/W)
F01	Task Creation and Editing	Allows users to create, edit, delete, and mark daily to-do tasks as completed.	Must
F02	Cross-Platform Local Storage (Offline-First)	Automatically saves tasks and user settings locally on the device (using SQLite), ensuring the app works seamlessly even without an internet connection.	Must
F03	Priority and Color Coding	Enables users to categorize tasks by	Must

		priority level (e.g., High, Medium, Low) with visual color tags.	
F04	Quick Search & Keyword Filtering	Allows users to instantly search through their entire task list using keywords.	Could
F05	Daily Task List & Status Filtering	Displays daily tasks cleanly in a unified UI and allows filtering by status (All, Active, Completed) or priority.	Must
F06	Cloud Database Synchronization	Automatically syncs task data between the user's PC and mobile device via a cloud database backend (e.g., Supabase) when online.	Should
F07	Visual Task Progress Tracking	Provides a dynamic progress bar or completion percentage reflecting daily task progress to keep users motivated.	Must

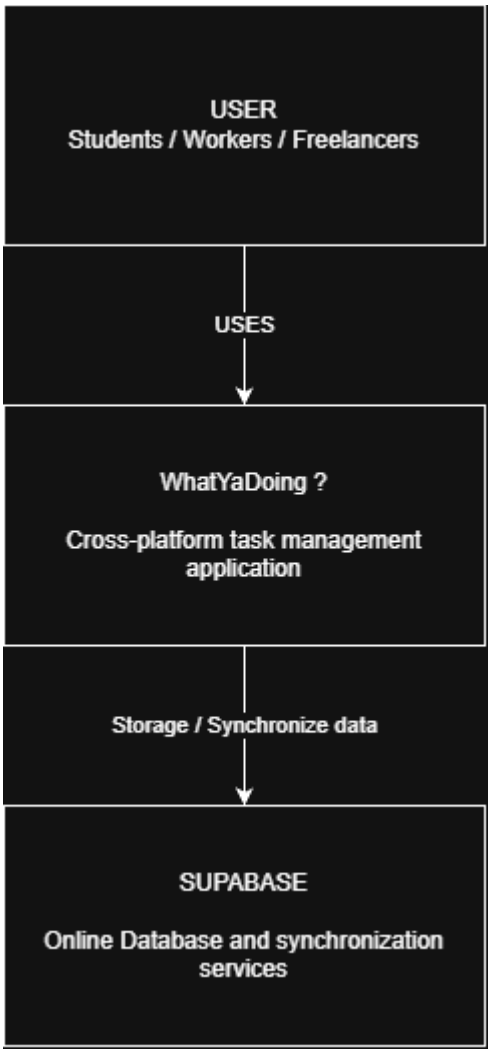
Out of scope:

✔️ *Real-time multi-user collaboration: Sharing tasks and collaboration between many individuals.*

3 Proposed Solution

3.1 Preliminary Architecture

[Draw a C4 System Context diagram: the system, its users and external systems (payment gateway, email service, ...). Detailed architecture is presented in PA3.]



3.2 Technology Stack

Layer	Technology	Reason for choice
Front end	reactnative + nativewind	React Native allows the application to share a common codebase across desktop and mobile platforms. Tailwind-style styling helps maintain a consistent and responsive

		user interface across different screen sizes.
Back end	Js + TypeScript + Express	JavaScript and TypeScript provide the main programming environment for the back end, while Express provides a lightweight framework for building the application's server-side logic and APIs.
Database	Supabase	Supabase provides a managed database and synchronization infrastructure that allows user data to be stored online and accessed across supported devices. It also supports the application's online functionality while local storage can be used to maintain offline access.
Hosting / deployment	github actions	GitHub Actions can automate the building and packaging of the application for supported platforms. This allows the team to consistently generate installable versions for Windows and Android.
Testing & CI	Jest	Jest provides automated testing for the application's core functionality and helps the team identify errors and regressions during development.

3.3 Hardware and Operating Environment

[Describe the hardware, devices and environments needed to develop, test and run the software.]

1. Development Environment

- **Developer Workstations:** Developers will use Windows, macOS, or Linux computers depending on their development setup.
- **Hardware Specifications:** A minimum of 8 GB RAM, a modern multi-core processor, and sufficient SSD storage are recommended. 16 GB RAM is recommended when running development tools and Android emulators simultaneously.
- **Development Tools:** The team will use Node.js, JavaScript/TypeScript, React Native, NativeWind, Express, Jest, Git, and an appropriate IDE or code editor.
- **Desktop Development:** Windows development and builds will be performed using the appropriate React Native Windows development tools and Visual Studio.
- **Mobile Development:** Android emulators and physical Android devices will be used to develop and test the mobile version of the application.
- **Database and Synchronization:** Supabase will be used to store application data online and synchronize data between supported devices.
- **Version Control:** Git and GitHub will be used to manage the source code and support collaboration between team members.

2. Testing Environment

- **Desktop Testing:** The application will be tested on supported Windows computers to verify its functionality, user interface, and performance.
- **Mobile Testing:** The application will be tested using Android emulators and physical Android devices where available.
- **Core Feature Testing:** Testing will cover task creation, editing, deletion, completion, priority assignment, task filtering, and daily progress tracking. These features are identified as the project's must-have functionality. PA0-ProjectIdea-TeamContract
- **Offline Testing:** The application will be tested without an Internet connection to verify that users can continue managing their available task data.
- **Online Testing:** The application will be tested with an Internet connection to verify communication with the Express back end and Supabase.
- **Synchronization Testing:** Changes made on one supported device will be tested to ensure that they are correctly synchronized and available on another supported device.
- **Automated Testing:** Jest will be used for automated testing of application functionality, while GitHub Actions will run tests automatically when changes are pushed to the repository.

3. Production Environment

- **Windows Deployment:** The Windows version will be packaged as an installable application so users can install and run it directly on their computers.
- **Android Deployment:** The mobile version will be packaged as an Android application that users can install on supported Android devices.
- **Back-end Services:** The Express back end will provide the required server-side functionality and communicate with Supabase for data storage and synchronization.
- **Online Data:** Supabase will store user data and allow the same user's task information to be accessed across supported devices.
- **Offline Operation:** The application will maintain locally available task data so users can continue using core task-management functionality when an Internet connection is unavailable.
- **Online Synchronization:** When an Internet connection becomes available, changes made locally will be synchronized with the user's online data.
- **Application Updates:** New versions will be built through the project's deployment pipeline and distributed as updated application packages for the supported platforms.
- **CI/CD:** GitHub Actions will automate testing and application builds to help ensure that new versions are tested before being distributed to users.

4 Project Plan

4.1 Process and Phase Gates

The project follows the waterfall model. Each phase ends with a phase gate (a PA submission). After a PA is submitted it becomes a baseline; later changes must be updated in the document and listed in the next PA.

PA	Phase	Due	Exit criteria (fill in)
PA1	Initiation & planning	End of week 3	The project proposal (PA1) is submitted to and approved by the TA.
PA2	Requirements	End of week 5	The Software Requirements

			Specification (SRS), including problem definition, target users and finalized feature list, is completed and approved.
PA3	Design	End of week 7	System architecture, class/component diagrams and UI wireframes are fully documented and approved.
PA4	Test planning	End of week 8	Test cases, test scenarios, and the test plan document are fully outlined and ready for execution.
PA5	Implementation, testing & deployment	End of week 10	All Must-have features are coded, unit/integration testing is executed with bugs resolved, and the executable desktop app is deployed.
Final	Presentation & demo	Week 11	The final product demonstration and project presentation are delivered and presented to the class.

4.2 Work Breakdown Structure (WBS)

1.0 Requirements and Planning Phase

1.1 Project Proposal & SRS Documentation

- **1.1.1** Define problem statements, target user groups, and use cases.
- **1.1.2** Document functional and non-functional requirements.
- **1.1.3** Finalize the main feature list and prioritize must-have items.

1.2 Project Management & Setup

- **1.2.1** Initialize version control repositories and TypeScript workspace structure.
- **1.2.2** Configure the task board and set up team tracking workflows.

2.0 System Architecture and Design Phase

2.1 Database & Sync Architecture Design

- **2.1.1** Design the database schema to support cross-device task syncing.
- **2.1.2** Define REST API routes & sync conflict resolution strategy.

2.2 User Interface & Component Design

- **2.2.1** Create wireframes and UI mockups for the desktop application layout.
- **2.2.2** Create wireframes and UI mockups for the mobile application layout.
- **2.2.3** Design shared TypeScript interfaces & API payload schemas.

3.0 Implementation Phase

3.1 Backend & Database Sync Service

- **3.1.1** Set up Express server, ORM/DB connection & middleware.
- **3.1.2** Build Express sync REST endpoints (GET/POST/PUT) & sync logic.

3.2 PC Client Development

- **3.2.1** Initialize PC application shell and window routing.
- **3.2.2** Implement task creation, editing, deletion, and local state management views for PC.
- **3.2.3** Implement UI components for progress tracking bars, priority color tags and keyword search filters for PC.

3.3 Mobile Client Development

- **3.3.1** Initialize Mobile application shell and navigation
- **3.3.2** Implement task creation, editing, deletion, and local state management views for mobile.
- **3.3.3** Implement UI components for progress tracking bars, priority color tags and keyword search filters for mobile.

4.0 Testing and Deployment Phase

4.1 Quality Assurance & Testing

- **4.1.1** Write unit tests for TS models & Express controller logic
- **4.1.2** Execute end-to-end integration testing for cross-device sync.

4.2 Deployment & Release

- **4.2.1** Build, package, and generate desktop installer executable.
- **4.2.2** Build and package the mobile application release bundle.

4.3 Estimation

Three-Point Estimation: $E = \frac{O + 4\ M + P}{6}$

WBS ID	Work package	Owner	Effort (person-hours)	Method / assumptions
1.1.1	Define problem statements, target user groups, and use cases	PM	6	Expert Judgment: Standard initial requirements breakdown.
1.1.2	Document functional and non-functional requirements	BA	8	Three-Point Estimate: $O = 6$, $M = 8$, $P = 12$

1.1.3	Finalize main feature list and prioritize must-have items	PM	4	Expert Judgment: Alignment with PA limits and team scope
1.2.1	Initialize monorepo & TypeScript workspace structure	Developer	5	Expert Judgment: Setup for PC and Mobile.
1.2.2	Configure task board, milestones, and tracking workflows	PM	3	Expert Judgment: Tool setup and task allocation.
2.1.1	Design database schema for cross-device task syncing	Architect	10	Three-Point Estimate: $O = 8$, $M = 10$, $P = 14$
2.1.2	Define REST API routes & sync conflict resolution strategy	Architect	8	Three-Point Estimate: $O = 6$, $M = 8$, $P = 12$
2.2.1	Create wireframes and UI mockups for PC application	Designer	8	Three-Point Estimate: $O = 6$, $M = 8$, $P = 12$
2.2.2	Create wireframes and UI mockups for Mobile application	Designer	8	Three-Point Estimate: $O = 6$, $M = 8$, $P = 12$
2.2.3	Design shared TypeScript interfaces & API payload schemas	Developer	6	Expert Judgment: Interface definitions shared across platforms.
3.1.1	Set up Express server, ORM/DB connection & middleware	Developer	8	Three-Point Estimate: $O = 10$, $M = 8$, $P = 6$
3.1.2	Build Express sync REST endpoints (GET/POST/PUT) & sync logic	Developer	6	Three-Point Estimate: $O = 5$, $M = 6$, $P = 7$

3.2.1	Initialize PC application shell and window routing	Developer	8	Expert Judgment: Framework boilerplate and window layout.
3.2.2	Implement task creation, editing, deletion, and local state management views for PC.	Developer	5	Three-Point Estimate: $O = 10$, $M = 4$, $P = 4$
3.2.3	Implement UI components for progress tracking bars, priority color tags and keyword search filters for PC.	Developer	6	Three-Point Estimate: $O = 5$, $M = 6$, $P = 7$
3.3.1	Initialize Mobile application shell and navigation	Developer	5	Expert Judgment: Mobile navigation and view routing.
3.3.2	Implement task creation, editing, deletion, and local state management views for mobile.	Developer	5	Three-Point Estimate: $O = 10$, $M = 4$, $P = 4$
3.3.3	Implement UI components for progress tracking bars, priority color tags and keyword search filters for mobile.	Developer	6	Three-Point Estimate: $O = 5$, $M = 6$, $P = 7$

4.1.1	Write unit tests for TS models & Express controller logic	Tester	6	Three-Point Estimate: $O = 7$, $M = 7$, $P = 5$
4.1.2	Execute end-to-end integration testing for cross-device sync	Tester	7	Three-Point Estimate: $O = 4$, $M = 7$, $P = 10$
4.2.1	Build, package, and generate desktop installer executable	Developer	4	Analogy: Standard desktop build pipeline configuration.
4.2.2	Build and package mobile application release bundle	Developer	4	Analogy: Standard mobile release build configuration.

4.4 Schedule

[Present a Gantt chart with task dependencies and milestones (the PA deadlines). Identify the critical path. Remember: effort ≠ duration.]

4.5 Roles and Responsibilities

Role	Main	Backup	Responsibilities
Team lead/ Project manager	Nguyễn Lương Hoàng Duy	Niê Ngọc Lâm	Plan, assign tasks, weekly status report, contact point
Business analyst	Niê Ngọc Lâm	Nguyễn Lương Hoàng Duy	Elicit and document requirements, organise

			reviews
Designer / Architect	Trần Phúc Đạt	Nguyễn Lương Hoàng Duy, Nguyễn Việt Quang	Architecture, class, data and UI design
Developers	Niê Ngọc Lâm	Nguyễn Việt Quang, Trần Phúc Đạt, Niê Ngọc Lâm, Nguyễn Lương Hoàng Duy	Implement, unit test, review pull requests
Tester	Nguyễn Việt Quang	Trần Phúc Đạt	Test plan, test cases, system testing, test report

4.6 Human Resources and Cost Estimate

[Estimate the total cost: effort × hourly rate (assume a realistic junior-developer rate) + tools, hosting and other costs.]

Category / Role	**Total Effort (Hours)**	**Hourly Rate ($/hr)**	**Total Cost ($)**
Project Management (PM)	13	25.00	325.00

Business Analysis (BA)	8	25.00	200.00
System Architecture	18	25.00	450.00
UI/UX Design	16	25.00	400.00
Development (PC + Mobile + Express)	68	25.00	1,700.00
Quality Assurance / Testing	13	25.00	325.00
Labor Cost	**136**	**—**	**3,400.00**

For the purpose of demo, the free tiers of the tools and services used are sufficient without needing to subscribe to a paid plan. Therefore, the additional cost for tools and services is assumed to be zero or negligible.

$$Total\ Cost = Labor\ Cost + Additional\ cost = 3,400 + 0 = 3,400\ (\$)$$

5 Risk Management

[Identify at least 5 project risks (technology, people, organisational, requirements, estimation). Risk exposure RE = Probability × Loss (use the scales from the Project Management lecture).]

ID	Risk	Type	Probability	Lo	RE	Mitigation / contingency	Owner
R1	Delays in one project phase may affect the next phases and cause the team to miss PA deadlines.	Schedule / Organisational	Occasional (50)	Serious (75)	3,750	Mitigation: Check task progress every week and identify important tasks. Contingency: Adjust the schedule, focus on tasks that affect deadlines, and postpone less important features if needed.	Team Lead
R2	Errors may occur when integrating the backend, database, PC app, and mobile app. These errors may prevent features from working correctly and delay development.	Technology	Likely (75)	Serious (75)	5,625	Mitigation: Agree on data structures and API rules before development. Integrate components early and test them regularly. Contingency: Fix errors affecting Must-have features first and postpone non-essential features if needed.	Developer / Designer-Architect
R3	Development delays or unexpected bugs may leave insufficient time to test, fix errors, and package the desktop app before the PA5 deadline.	Schedule / Quality	Likely (75)	Serious (75)	5,625	Mitigation: Test features during development and prepare the packaging process early. Contingency: Fix critical bugs first and complete all Must-have features for the desktop app before the deadline.	Tester / Developer
R4	Some application features or libraries may behave differently across Windows and Android, or the application may work correctly in the development environment but fail on	Technology/ Compatibility	Occasional (50)	Serious (75)	3,750	Mitigation: Test core features and packaged application releases on both Windows and Android. Verify platform compatibility, operating system requirements, permissions, and dependencies before	Developer / Tester

	end-user devices due to differences in operating system versions, device configurations, permissions, or runtime dependencies. This may prevent the application from running correctly on supported platforms.					deployment. Contingency: Identify and fix platform-specific or environment-related issues, and replace incompatible libraries or adjust configurations when necessary.	
R5	Incorrect database permissions or insufficient access control in the backend may allow unauthorized users to access or modify task data. This could compromise user privacy and data integrity.	Technology / Security	Seldom (25)	Serious (75)	1,875	Mitigation: Configure appropriate database access policies and verify that users can only access authorized data. Keep secret API keys in secure backend configuration and never expose them in client code. Contingency: Restrict affected access, correct the security configuration, and replace any exposed secret keys if necessary.	Developer
R6	Insufficient communication and incomplete task handoffs between team members may lead to misunderstandings about requirements, design decisions, or implementation details. This may cause duplicated work, rework, and delays in completing project tasks.	People / Communication	Occasional (50)	Moderate (50)	2,500	Mitigation: Record important technical decisions, keep task status updated on the task board, and review interfaces and requirements before handing work over to another team member. Contingency: Clarify the agreed requirements and responsibilities, correct affected work, and update related tasks to prevent further misunderstandings.	Team Lead / Project Manager

6 Tools & Collaboration Setup

One of the main goals of the project is to practise teamwork for professional software development. The use of tools is monitored and graded (teamwork component). The team must use:

✔️ Moodle – announcements and PA submissions.

✔️ Communication – Slack, MS Teams or Discord: one channel per team; the TA is a member.

✔️ Task board – GitHub Projects (recommended) or Jira: every WBS work package is a task with an owner and due date.

✔️ Git repository – GitHub (recommended), GitLab or Bitbucket; the instructor and TA have access:

- main branch is protected; changes are merged through pull requests with at least one review
- README (description, setup), .gitignore; never commit secrets (.env, API keys, passwords)
- commit messages follow the Conventional Commits style (feat:, fix:, docs:, test:, …)

✔️ Weekly status meeting (15 minutes) and weekly status report (template provided), committed to /docs/management/weekly every week from week 2 to week 10. The team chooses its weekly deadline and states it here.

Repository structure:

Folder	Content
/src	Source code

/docs/management	Project plan; weekly status reports in /docs/management/weekly
/docs/requirements	Vision, SRS, use cases, mockups
/docs/design	Architecture (C4), ADRs, UML models, UI design
/docs/test	Test plan, test cases, test reports
/docs/ai-usage	AI usage logs
/pa/PA1 … /pa/PA5	One sub-folder per PA submission (PDF + source files)

7 AI Usage Log

[Required in every PA (course GenAI policy). List each significant use of AI tools. Write "No AI tools were used" if none. Undisclosed AI use is treated as an academic integrity violation.]

#	Member	Tool	Task / purpose	How the output was verified and changed
1	Nguyễn Lương Hoàng Duy	Gemini	Reviewing and refining section 4.2, 4.3, 4.4 and 4.6	Discussion with the BA on the soundness of the planning after AI review and refine. Several changes were made after the BA put forward some suggestions and the final draft was finalized after notifying other team members.
2	Trần Phúc Đạt	ChatGPT, Gemini	Assisting with drafting and refining Section 1: Proposed Solution, including the C4 System Context diagram, technology stack, and hardware and operating	Reviewed the AI suggestions against the team's project requirements and revised the output to match the planned React Native and TypeScript implementation, Windows and Android support, Supabase database, offline functionality, online synchronization, and platform-specific deployment.

			environment for the cross-platform WhatYaDoing? application.	
3	Nguyễn Việt Quang	ChatGPT	Filter and choose main features for the products in section 2.4	Analyze team member capabilities, selects and adjust core features.
4	Niê Ngọc Lâm	ChatGPT	Assisting with reviewing the technology stack in Section 3.2 and drafting Section 5: Risk Management, including identifying potential project risks and suggesting mitigation and contingency plans.	Reviewed AI suggestions against the project's technology stack, work breakdown structure, and project plan. Selected relevant risks, revised risk descriptions and mitigation strategies, and checked Probability, Loss, and Risk Exposure.